Yuvam Bhargav.

Assignment - 04.

1. [4 pts] In the algorithm we learned in class **SELECT(*A, i*)**, the input elements are divided into groups of 5. Will the algorithm work in linear time if they are divided into groups of 7? Argue that SELECT does not run in linear time if groups of 3 are used.

Ans.) Yes. I think the Algorithm would still be able to work, if we divide the elements into groups of 7.

Because :-

→ Even if we divide the elements into groups of 7, we will still be able to get a good pivot point that will again guarantee / give us a constant fraction (as we got in case of 5 elements $[n/4]$) to discard at every recursive call. It will also ensure a time complexity of $O(n)$. {Linear}.

⇒ SELECT would not be able to run in Linear Time if we divide the elements into group of 3. Because :-

→ We will not get a good pivot point as the Median-of-Medians method require good amount (slightly Larger) data / elements to reduce constant fraction (as we got in case of 5 elements $[n/4]$) at each recursive call.

→ If we divide the elements into groups of 3, the recurrence eqn would become, $T(n) = T(n/3) + T(2n/3)$ which will provide a time complexity :- $O(n \log n)$. [Worse and No longer Linear.]

2. [2 pts] Consider the following recursive function, write the recurrence equation, solve it, and answer what is the time complexity of this function (in terms of big-O notation)?

```
int funA(int n)
{
        if (n <= 1)
                return n;
        return 2*funA(n-1);
}
```

→ Recursion // $(n-1)$.

→ $\times 2$ // (multiply 2) is constant.

Ans.) Equation :-

→ One Recursive call and multiplication by 2 (constant)

⇒ $T(n) = T(n-1) + 1$ ≥

⇒ $(T(n-1) + 1) + 1 \ldots\ldots$ ⇒ $O(n)$. {Linear}.

★ Time Complexity :- $O(n)$.

3. [8 pts] Assume that you need to keep track of some data. To store those data, you have the following options:

A) A linked list maintained in sorted order. // Linked List → Sorted.
B) A linked list of unsorted records. // Linked List → Unsorted.
C) A binary search tree. // BST.
D) An array maintained in sorted order. // Arr. → Sorted.
E) An array of unsorted records. // Arr. → Unsorted.

For each of the following scenarios, which of the above choices would be the best (*i.e.* you will pick)? Justify your answer.

3.1) The records are guaranteed to arrive in sorted order from lowest to highest (*i.e.*, whenever a record is inserted, its key value will always be greater than that of the last record inserted). A total of 1000 inserts will be interspersed with maximum 1000 searches. // Sorted, 1000 searches + Insertions.

Ans.) [D.] Array maintained in Sorted order. (Sorted Arr.)

Reason :-
→ Inserted as the last element in an array (Sorted order assured Lowest to Highest) ⇒ O(1).

·) For BST :- O(log n)
·) Others require various Traversal steps.

3.2) The records arrive with values having a uniform random distribution (*i.e.*, the BST is likely to be well balanced in case you want to use it). 1,000,000 insertions are performed, followed by 10 searches. // Random, 1000000 insertions & 10 Searches.

Ans.) [E]. An array of unsorted Records.

→ Has only 10 searches but massive in terms of Data/insertions. It would be best to use an Unsorted Array or even an Unsorted Linked List where the insertions would be done very quickly. ⇒ O(1).

3.3) The records arrive with values having a uniform random distribution (*i.e.*, the BST is likely to be well balanced in case you want to use it). 1000 insertions are interspersed with 1000 searches.

Ans.) [C]. Binary Search Tree. (BST). // Random, 1000 insertions + 1000 Searches.

→ Searching and Inserting simultaneously ⇒ O(log n) [Avg. Case] for each step/operation.

→ A Balanced BST would be much more effective than Arrays or Linked Lists.

3.4) The records arrive with values having a uniform random distribution (*i.e.*, the BST is likely to be well balanced in case you want to use it). 1000 insertions are performed, followed by 1,000,000 searches.

// Random, 1000 insertions, then 1000000 Searches.

Ans.)

[C] Binary Search Tree.

→ Again, Insertions and Searches can be done effectively using a Balanced BST ⇒ O(log n).

4. [6 pts] Given an array of n distinct integers, we consider the following 3 different algorithms to find the top-k largest elements:

- Algorithm #1: Use Selection Sort algorithm to partially sort the array until the first k elements are in their correct positions. // Selection Sort. → Partially Sort (First k items) ⇒ O(n.k).
- Algorithm #2: Sort the entire array using Merge Sort and then extract the top k elements. // MERGE SORT (Ann.) and select k items ⇒ O(n log n).
- Algorithm #3: Use the SELECT algorithm to find the nth, (n-1)th, (n-2)th, …, (n-k)th ranking elements one by one. // SELECT Sort → should apply k times, find greatest elements.

Compare the time complexities and performance of all 3 algorithms for different values of k. Explain which approach is more efficient for each scenario and why.

- when k is much smaller to n and can be viewed as a constant to n. *i.e.* $k = O(1)$
- when k is close to n, *i.e.* $k = O(n)$.

Ans.) top-K Largest element :-

CASE-1 :- K is much smaller to n. → K = O(1) // Small K.

★ Algorithm 1 ⇒ O(n) // K is constant.

★ Algorithm 2 ⇒ O(n log n)

★ Algorithm 3 ⇒ O(n) // For each SELECT.

→ Total = O(n).

CASE-2 :- K is close to n. → K = O(n). // Larger K.

★ Algorithm 1 ⇒ $O(n^2)$.

★ Algorithm 2 ⇒ O(nlogn).

★ Algorithm 3 ⇒ $O(n^2)$ // Repetitive SELECT.

By this, I think :-

→ For small K, Case 1 [O(1), Constant], Algorithms 1 and 2 will be efficient. // O(n).

→ For Larger K., K ≈ n [O(n)], Algorithm 2 will be efficient. // O(nlogn).

5. [5 pts] Given a set S of n distinct numbers and a positive integer k, assume $k \ll n$ (much smaller than n, *i.e.* can be viewed as a constant compare to n), we consider the problem of finding k numbers in S that are closest to the median of S. Describe your algorithm clearly and analysis its running time in big-O notation.

Ans.) Given :-

$S = \{ n$ distinct members, +ve int k. where $k << n$ (constant)$\}$

To Find => Algorithm to find k numbers within 'S' closest to Median. # With Time Complexity

→ We can use Median-of-Medians method (SELECT), used in the class to find a median value for the set S. [Let that value be m.]. } O(n). // Linear.

→ After finding the median m, we can try to iterate it over the set again and try to calculate the divergence / Difference between each element and the median m. { Say, $|x - m|$, where x is element in Set S. }

And Store this data in an array. ("Difference", "element"). } O(n). // Linear.

→ I think now we can again use Median-of-Medians method over this array to get the k closest number of medians. } O(n). // Linear.

* Complexity :-

=> $O(n) + O(n) + O(n)$.

=> $T(n) = O(n)$. // Linear.

#=> I think this will be an efficient Algorithm as it has Linear Runtime.

6. [5 pts] Let $A[1 \ldots n]$ and $B[1 \ldots n]$ be two arrays, each containing n numbers that are already sorted in order. Describe an algorithm to find the median of all the $2n$ elements in Arrays A and B. Analysis your algorithm's running time in big-O notation.

Ans.) Given :-

→ 2 Arrays. => $A = [1 \ldots n]$; $B = [1 \ldots n]$ // Sorted.

To Find :-

Algorithm to find median of all 2n elements in Arr. A and B.

With Time Complexity.

→ Arr A and B are equal in size and are sorted. So I think we can try performing Binary search on these sorted Arrays.
(By having 2 pointers to Binary Search partition.).

→ This will divide the total (2n) elements from both the Arrays into 2 parts. We can call them Left and Right partitions both having n elements to sum up to 2n elements.

O(1). // To Access these Arrays

⇒ This will also provide us with all the smaller elements to the left partition. Basically all the elements in the Left partition would be smaller than the elements in Right partition. // from both the Arrays

★ Pointers :-

→ i for array A.
→ j for array B. (n-i). } So total elements in Left side remains n. // including Both the Arrays.

O(nlogn). // No. of Iterations.

• Partition ↳ A[i] ≥ B[j-1] && B[j] ≥ A[i-1]. // This will ensure correct Partition.

→ After separating them / partitioning, the median m would be :-

Average of
⇒ Largest element from left.
And
⇒ Smallest element from Right.

For total 2n elements :

$$\frac{\text{Max}(A[i-1], B[j-1]) + \text{Min}(A[i], B[j])}{2}$$

★ Time Complexity :-

~> Constant time for comparing = O(1). // each step.

~> Total number of iterations / steps = O(logn).

$$T(n) = O(\log n).$$

#> ∴ I think this Algorithm will efficiently accomplish the Required Task, with a Runtime = O(logn).